

AILANG Display System — Complete Design Document v2.0

BSP Stack+Tile Compositor with Four-Ring Event Architecture

Author: Sean Collins, 2 Paws Machine and Engineering

License: SCSL

Status: Design / Pre-implementation

Revision: v2.0 — Incorporates synchronization, event handling, BSP tiling, deskbar, hard boundaries, hotkeys

1. Problem Statement

Building a GUI in C/C++ today requires:

- Linking against Qt (500MB+), GTK, or similar monolith
- Build systems: CMake, qmake, meson, pkg-config
- Preprocessors: MOC (Qt), Vala (GTK)
- ABI compatibility nightmares between compiler and library versions
- Header files pulling in hundreds of transitive includes
- Understanding 47 virtual methods on QWidget just to show a rectangle
- Transparency compositing burning CPU/GPU for visual gimmicks
- Multi-desktop workflows that hide windows instead of organizing them
- Off-screen drawing bugs that lose windows past monitor edges

The AILANG answer: Write your GUI code at the level you need it. The display system is a service you talk to, not a library you link against. Same pattern as the syscall layer — flat calls, handles in/out, each axis independent. Hard boundaries everywhere. One desktop. Stack+tile by default.

2. Design Philosophy

2.1 Six Orthogonal Axes

Each axis is independently usable. You never need axis N+1 to use axis N.

Axis 0: Surface — rectangular buffer (pixels or cells)
Axis 1: Draw — put shapes/text/images into a surface
Axis 2: Compose — BSP region map, stack groups, zone output
Axis 3: Input — four-ring event architecture
Axis 4: Layout — compute rectangles from constraints
Axis 5: Widget — OPTIONAL convenience glue (uses 0-4)

2.2 Core Invariants

These are non-negotiable. Every component enforces them.

INVARIANT 1: No surface pixel exists outside its allocated dimensions
INVARIANT 2: No window position places any part outside the screen bounds
INVARIANT 3: The deskbar's thin band is excluded from all window positioning
INVARIANT 4: BSP leaf dimensions == surface dimensions (always, no exceptions)
INVARIANT 5: Draw operations that would exceed surface bounds are clamped/rejected
INVARIANT 6: Apps never know their screen position, only their surface dimensions

2.3 Core Rules

1. **Surface is the universal primitive** — everything draws into surfaces, compositor reads surfaces, input routes to surfaces
2. **Format dispatch, not backend dispatch** — the surface format determines behavior. Terminal cell surfaces and pixel surfaces can coexist
3. **No inheritance, no objects** — widgets are {descriptor + 3 function pointers}, windows are {surface + decorator + layout + input focus}
4. **Sub-surfaces for clipping** — sub-surface boundaries ARE the clip region, no separate clip stack
5. **Hard boundaries** — surfaces have fixed dimensions, nothing draws outside them, no off-screen content
6. **No transparency in compositing** — surfaces are opaque. The compositor copies, never blends. Alpha blending exists only WITHIN a surface for app-level use (sprites, etc.)
7. **Everything independently testable** — layout can be unit tested without drawing a pixel
8. **Syscall-style calls** — flat functions, handles in/out, no method dispatch
9. **Client/server split** — apps talk to display server via shared memory + command pipe. Same API works standalone (direct calls) or multi-process
10. **Zero-copy surfaces** — shared memory means client draws directly, server composites from same memory
11. **Display server is an AILANG binary** — same compiler, same syscall layer, one stack all the way down

12. **One desktop** — no virtual desktops. Stack+tile solves the window management problem that virtual desktops badly approximate
 13. **Deskbar is king** — the deskbar's reserved band is a hard boundary. Nothing draws under it.
-

3. No Transparency — Design Rationale

3.1 What Transparency Costs

Every frame with transparency requires back-to-front rendering through every overlapping layer, blending each pixel: $\text{out} = \text{src} * \alpha + \text{dst} * (1 - \alpha)$. That's a multiply-add per channel per pixel per layer. For three overlapping windows on 1920×1080, that's potentially 24 million blend operations per frame.

Transparency poisons the dirty region system. Without it, a dirty rect belongs to exactly one window — the topmost covering that region. With transparency, a dirty rect at the bottom of the z-stack forces recomposite of every layer above it. $O(1)$ becomes $O(\text{layers})$ per dirty rect.

Hit-testing becomes ambiguous. With opaque windows, every pixel belongs to exactly one window. With transparency, click routing is undefined.

3.2 What Replaces It

Stack+Tile from Haiku OS. The two things people actually want from overlapping windows are:

- "Switch between related windows quickly" → **Stack** (tabbed window groups)
- "See two things side by side" → **Tile** (edge-snapped windows)

Neither requires transparency. Neither requires overlap at all in the tiled case.

3.3 Compositor Simplification

```
// TRANSPARENCY VERSION (killed)
```

```
for each layer back-to-front:
```

```
  for each pixel in layer's visible region:
```

```
    dst = read output pixel
```

```
    src = read layer pixel
```

```
    alpha = layer opacity * src alpha
```

```
    out = blend(src, dst, alpha)
```

```
    write output pixel
```

```
// OPAQUE VERSION (what we use)
```

```
for each dirty region:
```

```
owner = BSP_Query(region)
memcpy(output + region_offset, owner.surface + surface_offset, region_size)
```

A memcpy per dirty region. No blending, no back-to-front ordering for rendering, no per-pixel math.

3.4 What We Lose (And Why That's Fine)

- Drop shadows between windows — gone
- Translucent panel backgrounds — gone
- Fade-in/fade-out animations — possible as surface-level effects (app draws its own fade internally, compositor doesn't care)
- Desktop widgets floating semi-transparently — either opaque or they don't exist

Good riddance. Every one is a gimmick that costs real performance and real code complexity.

4. Multi-Monitor Architecture

4.1 Unified Surface With Zones

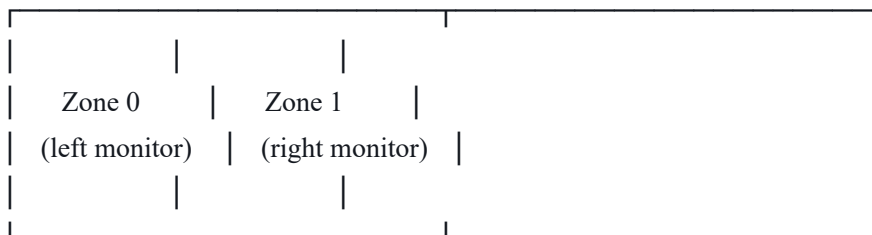
The screen IS one giant surface. Multiple monitors are regions within it. Each physical monitor is a "zone" — a rectangle within the unified surface that maps to an output device.

Example: Two 1920×1080 monitors side by side

Unified surface: 3840×1080

Zone 0: x=0 y=0 w=1920 h=1080 output=connector0

Zone 1: x=1920 y=0 w=1920 h=1080 output=connector1



One surface: 3840 × 1080

4.2 Two Surface Modes

Mode 0 — Combined: All monitors become one zone. One BSP tree, one region map, windows tile freely across the whole surface. The compositor renders one surface and the output stage splits the blit across devices.

This is the zero-bezel ultrawide case.

Mode 1 — Zoned: Each monitor is an independent zone. Each zone gets its own BSP tree root. Windows tile within their zone by default. Dragging a window across the zone boundary pulls it from one BSP, makes it temporarily floating, and snaps it into the target zone's BSP on drop. The zone boundary is a soft wall for drag operations.

The unified surface always exists in both modes. The mode controls whether there's one BSP root or multiple, and whether tiling constraints cross zone boundaries by default.

```
FixedPool.SurfaceMode {
    "COMBINED": Initialize=0
    "ZONED":    Initialize=1
}

FixedPool.DisplayConfig {
    "surface_mode": Initialize=0
    "surface_width": Initialize=0 // total unified width
    "surface_height": Initialize=0 // total unified height
    "zone_count":    Initialize=0
    "zone_table":    Initialize=0 // pointer to zone array
    "bsp_root":      Initialize=0 // combined mode: one root
    "zone_bsp_roots": Initialize=0 // zoned mode: array of roots
}
```

4.3 Zone Configuration

Zones don't have to be horizontally adjacent. The x,y offset for each zone is set based on physical arrangement. Monitor above? Zone 1 gets a negative y offset. Weird diagonal? Set whatever x,y makes mouse movement feel right.

```
FixedPool.ZoneFields {
    "X":      Initialize=0 // position in unified surface
    "Y":      Initialize=0
    "WIDTH":   Initialize=0 // physical resolution
    "HEIGHT":  Initialize=0
    "OUTPUT_DEVICE": Initialize=0 // render backend handle
    "REFRESH_RATE": Initialize=0 // for frame pacing
    "SCALE_NUM": Initialize=1 // DPI scale numerator
    "SCALE_DEN": Initialize=1 // DPI scale denominator
    "LOGICAL_WIDTH": Initialize=0 // WIDTH * DEN / NUM
    "LOGICAL_HEIGHT": Initialize=0 // HEIGHT * DEN / NUM
}
```

4.4 DPI Scaling

DPI scaling lives at the zone level because you might have a 4K monitor next to a 1080p monitor. Integer ratio scaling, not floating point.

- 1:1 for 1080p standard
- 2:1 for 4K at 1080p logical size
- 3:2 for intermediate cases

The BSP and layout system work in **logical coordinates**. The render stage scales to physical coordinates per-zone. A window that's 960×540 logical on a 2:1 zone blits to 1920×1080 physical pixels.

Surface buffers are allocated at physical resolution. DPI-aware apps draw at full resolution. DPI-unaware apps draw at logical resolution and the compositor nearest-neighbor scales up.

For terminal mode: cell size scales. A cell on a 2:1 zone is 16×32 pixels instead of 8×16. The font renderer picks the right glyph size. The BSP works in cell counts.

4.5 Render Pipeline With Zones

For each zone:

```
scale = zone.SCALE_NUM / zone.SCALE_DEN
```

For each dirty region intersecting this zone:

```
leaf = BSP_Query(region)
```

```
surface = leaf.stack_group-surfaces[active_index]
```

```
// Blit from surface to zone output
```

```
// scale 1:1 → memcpy
```

```
// scale != 1:1 → scaled blit (nearest neighbor)
```

```
ZoneBlit(zone, surface, region, scale)
```

```
// Blit any floating windows in this zone
```

For each float overlapping zone:

```
ZoneBlit(zone, float.surface, float_region, scale)
```

```
ZoneFlush(zone) // push to output device
```

5. BSP Stack+Tile Compositor

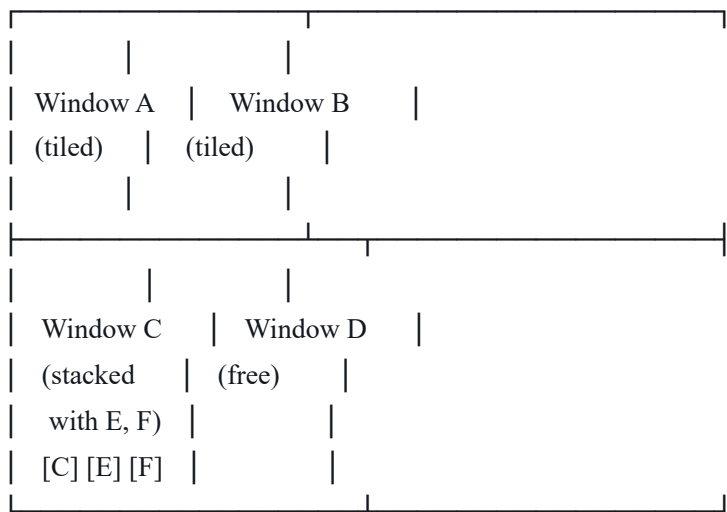
5.1 Overview

The compositor is a region manager, not a blender. Every pixel on screen belongs to exactly one surface. The compositor maintains a BSP tree that maps screen rectangles to surface owners. Compositing is a copy, not a blend.

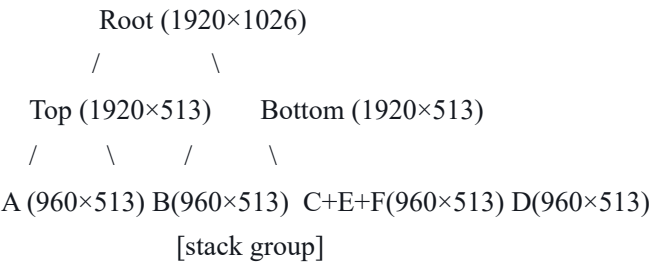
5.2 BSP Tree Structure

Every split is horizontal or vertical, mapping directly to tile edges. When two windows tile, that's a BSP split — the parent region divides into two children along the shared edge.

Example layout:



BSP tree:



5.3 Data Structures

```
FixedPool.BSPNodeType {
    "SPLIT_H": Initialize=0 // horizontal split (top/bottom children)
    "SPLIT_V": Initialize=1 // vertical split (left/right children)
    "LEAF": Initialize=2 // contains a stack group
}
```

```

FixedPool.BSPNodeFields {
    "TYPE":      Initialize=0  // SPLIT_H, SPLIT_V, or LEAF
    "PARENT":    Initialize=0  // parent node (0 = root)
    "REGION_X":  Initialize=0  // logical coordinates
    "REGION_Y":  Initialize=0
    "REGION_W":  Initialize=0
    "REGION_H":  Initialize=0
    // Split nodes only:
    "SPLIT_POS": Initialize=0  // position of split within region
    "SPLIT_MIN": Initialize=0  // minimum split position
    "CHILD_A":   Initialize=0  // left/top child
    "CHILD_B":   Initialize=0  // right/bottom child
    // Leaf nodes only:
    "STACK_GROUP": Initialize=0 // pointer to stack group
}

FixedPool.StackGroupFields {
    "SURFACE_COUNT": Initialize=0
    "SURFACES":      Initialize=0 // array of surface handles
    "ACTIVE_INDEX":  Initialize=0 // which tab is visible
    "TAB_NAMES":     Initialize=0 // array of title string pointers
    "BSP_LEAF":      Initialize=0 // back-pointer to owning BSP leaf
    "DECORATOR_ID":  Initialize=0 // which decorator draws the chrome
}

```

5.4 Stack Groups

A BSP leaf doesn't point to a single surface — it points to a stack group. A stack group holds one or more surfaces (tabs). Only the active surface is visible and composited.

- A window with no stacking = stack group with one entry
- Stacking adds entries to the group
- Switching tabs changes ACTIVE_INDEX, posts dirty flag for that region
- Inactive surfaces cost nothing — compositor only ever blits `SURFACES[ACTIVE_INDEX]`

5.5 Floating Windows

Floating windows exist during drag operations and before a window snaps into the BSP. They are transient.


```
FixedPool.FloatEntry {
    "SURFACE":    Initialize=0
    "X":          Initialize=0 // logical position
    "Y":          Initialize=0
    "W":          Initialize=0
    "H":          Initialize=0
    "Z_ORDER":    Initialize=0 // overlap among floats only
    "DRAGGING":   Initialize=0 // currently being repositioned
}

FixedPool.FloatConfig {
    "MAX_FLOATING": Initialize=8 // small — floats are transient
    "SNAP_THRESHOLD": Initialize=8 // logical px for edge snap detection
    "TAB_DROP_ZONE": Initialize=24 // height of title bar drop target
}
```

Floating windows are clamped to screen bounds at all times:

```
float_x = Max(0, Min(float_x, screen_width - float_w))
float_y = Max(deskbar_height, Min(float_y, screen_height - float_h))
```

A window can never leave the screen. The title bar is always reachable.

5.6 Hit Testing

Input routing priority:

1. Deskbar region (always first, deskbar is king)
2. Float list (z-ordered, top down — transient only)
3. BSP tree (tiled windows)

BSP hit test: $O(\log n)$

At split node: point on which side of SPLIT_POS? → recurse that child

At leaf: return STACK_GROUP.SURFACES[ACTIVE_INDEX]

Float scan: $O(\text{floats})$, max 8

Total worst case: $8 + \sim 10 = 18$ comparisons. Effectively instant.

5.7 Window Placement Lifecycle

New window creation:

Window created:

- IF auto_stack enabled (default):
 - add as tab to focused stack group, make it the active tab
- IF auto_stack disabled:
 - place floating at center of focused zone

User can then:

- leave it (it's a tab in the current stack)
- drag tab out (becomes floating)
- drag edge to another window's edge → BSP split (tiled)
- drag title to another window's tab area → join that stack

Snap gestures — two distinct actions:

Edge snap → BSP split (tiling): Drag window edge near another window's opposing edge. Within snap threshold → lock together → create BSP split on mouse-up. Both windows become BSP leaves sharing that edge as a constraint.

Tab-area drop → Stack: Drag window title bar onto another window's title/tab area. Larger window's dimensions become the stack group size. Newly dropped window becomes the active tab. Previous window becomes background tab.

BSP restructuring cases:

Case 1 — Both floating, edge snap:

Before: floating [A], floating [B]

After: BSP split node → leaf A | leaf B

Both removed from float list

Case 2 — One tiled, one floating, edge snap:

Before: BSP leaf [A], floating [B]

After: A's leaf splits → A one side, B other side

B removed from float list

Case 3 — Tab drop onto tiled window:

Before: BSP leaf [A], floating [B]

After: A's leaf stack becomes {A, B}, B is active

Leaf resizes to max(A, B) dimensions if needed

Case 4 — Tab drop onto existing stack:

Before: BSP leaf [stack: A, C active], floating [B]

After: Stack becomes {A, C, B}, B is active

6. Four-Ring Event Architecture

6.1 Overview

Four independent ring buffers handle synchronization and event dispatch. They are processed in order every tick, which prevents re-entrancy and callback chains (the Qt disease).

Ring 0: Focus/Control — lowest frequency, highest authority

Ring 1: Input Events — highest frequency, time-sensitive

Ring 2: Draw Commands — bursty, per-client, round-robin

Ring 3: Output — driven by dirty state, lowest urgency

6.2 Why Four Rings

The event queue (input) and command queue (draw) have different producers and consumers. Mixing them creates priority inversion — mouse-move floods starve draw commands, or vice versa.

Each ring has different overflow characteristics, different processing priority, and different frequency. Separating them means each interaction between subsystems is a well-defined post to another ring, not a tangled callback chain.

A click goes: Ring 1 (mouse event) → handler posts to Ring 0 (focus change) → *next tick* Ring 0 updates routing → Ring 2 (focused window redraws border) → Ring 3 (compositor flushes). Each step is one ring entry, processed in order, no re-entrancy.

6.3 Ring 0 — Focus/Control

Purpose: Policy decisions. Who owns what, z-order, focus, window lifecycle, BSP structure changes. These are rare but when they happen everything else must respect the new state immediately.

Processed first every tick because it changes the routing tables for Ring 1.

Typical throughput: 0-2 commands per tick. Usually empty.

Ring size: 32 entries. Should never overflow.

Overflow policy: Should never happen. If it does, block the poster (this indicates a bug).

```

FixedPool.Ring0Commands {
    // Window lifecycle
    "WIN_CREATE":    Initialize=1
    "WIN_DESTROY":   Initialize=2

    // Stack operations
    "STACK_ADD":     Initialize=10
    "STACK_REMOVE":  Initialize=11
    "STACK_SWITCH":  Initialize=12

    // Tile operations
    "TILE_SPLIT_H":  Initialize=20
    "TILE_SPLIT_V":  Initialize=21
    "TILE_MERGE":    Initialize=22
    "TILE_RESIZE":   Initialize=23

    // Focus
    "FOCUS_SET":     Initialize=30
    "FOCUS_MODE":    Initialize=31

    // Zone management
    "ZONE_ADD":      Initialize=40
    "ZONE_REMOVE":   Initialize=41
    "ZONE_RECONFIGURE": Initialize=42

    // Deskbar
    "DESKBAR_UPDATE": Initialize=50
    "DESKBAR_COLLAPSE": Initialize=51
    "DESKBAR_EXPAND": Initialize=52

    // Global
    "QUIT":          Initialize=99
}

```

6.4 Ring 1 — Input Events

Purpose: Raw keyboard, mouse, resize, timer events. High frequency, time-sensitive. Routing depends on Ring 0's current state.

Ring size: 64-128 entries. Mouse-move events come fast.

Overflow policy: Drop oldest mouse-move events. **Never** drop key events. Key events are lower frequency and losing one means a lost keystroke — unacceptable.

Routing logic:

For each event in Ring 1:

If mouse event:

- HitTest(x, y): check deskbar → floats → BSP
- route to target surface's app
- If click-to-focus mode and target != focused:
post FOCUS_SET to Ring 0 (handled next tick)

If keyboard event:

- If Alt held: check hotkey table
- If match: post action to Ring 0, consume event
- Else: route to focused window's app

If global hotkey (alt-tab, etc.):

- post to Ring 0, consume event

If resize:

- handle immediately (zone reconfiguration)

If quit:

- handle immediately

6.5 Ring 2 — Draw Commands

Purpose: In server mode, each client pushes draw commands. In standalone, widget handlers push draw operations. Commands mutate surfaces and set DIRTY flags.

Ring size: 256 per client.

Overflow policy: Back-pressure the client (block until space). A client that floods draw commands must wait.

Scheduling: Round-robin across clients. Each client gets N commands processed per tick. No single client can monopolize the compositor.

For each client in round-robin order:

- Process up to N commands from client's Ring 2
- Each command: mutate surface, set DIRTY flag
- If client has more pending: wait for next tick

6.6 Ring 3 — Output

Purpose: Not really a command ring — it's a work list. Dirty regions that need compositing and flushing. Populated by Ring 2 setting dirty flags, drained by the render phase.

Overflow: Not applicable. Driven by dirty flags, not queued commands.

Processing:

If any DIRTY flags set:

- Merge dirty regions (reduce draw calls)

For each dirty region:

- BSP query → find owning surface

- Blit to zone output (memcpy for scale 1:1, scaled blit otherwise)

- Blit any dirty floating windows

- Flush each affected zone to its output device

- Clear DIRTY flags

Skip entirely if nothing changed (zero-cost idle)

6.7 Ring Data Structure

All four rings use the same head/tail ring buffer pattern established in the existing mailbox system:

```
FixedPool.RingBuffer {
    "buffer": Initialize=0 // pointer to entry array
    "head":   Initialize=0 // read position
    "tail":   Initialize=0 // write position
    "capacity": Initialize=0 // ring size (power of 2)
    "count":  Initialize=0 // current entries
    "entry_size": Initialize=0 // bytes per entry
}
```

// Post to ring (producer)

```
Function.Ring_Post {
    Input: ring, entry_ptr
    // Check overflow per ring's policy
    // Copy entry to buffer[tail]
    // tail = Modulo(Add(tail, 1), capacity)
    // count = Add(count, 1)
}
```

// Drain from ring (consumer)

```
Function.Ring_Drain {
    Input: ring
    Output: entry_ptr or 0
    // If head == tail: return 0 (empty)
    // Read entry from buffer[head]
    // head = Modulo(Add(head, 1), capacity)
    // count = Subtract(count, 1)
}
```

6.8 The Tick Cycle

Every tick:

1. Drain Ring 0 (focus/control)
 - Process window create/destroy
 - Process BSP restructuring (split/merge/resize)
 - Process focus changes
 - Update routing tables
 - $O(1)$ most ticks (ring is usually empty)
2. Drain Ring 1 (input events)
 - Route mouse by hit-test against current BSP/float/deskbar state
 - Route keyboard to focused window
 - Intercept Alt+key for hotkeys → post to Ring 0
 - Mouse clicks may generate Ring 0 posts (click-to-focus)
 - Mouse events may trigger next-tick Ring 0 processing
3. Process Ring 2 (draw commands) — round-robin per client
 - Each client gets N commands per tick
 - Commands mutate surfaces, set DIRTY flags
 - Fair scheduling, no starvation
4. Process Ring 3 (output)
 - If DIRTY: merge regions, composite, flush zones
 - If clean: skip entirely
 - Zero-cost when idle
5. Yield / sleep until next event or frame deadline

7. Focus Policy

7.1 Focus Modes

The focus policy is pluggable, stored as a mode flag. The compositor supports three modes, switchable at runtime:

Mode 0 — Click to focus (DEFAULT):

Mouse click on a window raises z-order (for floats) and gives keyboard focus.

Mouse movement alone routes mouse events by hit-test but does NOT change keyboard focus. This is what most users expect.

Mode 1 — Focus follows mouse (sloppy focus):

Mouse entering a surface's region gives it keyboard focus.

Z-order doesn't change. Useful for tiling/power-user setups.

Mode 2 — Focus follows mouse + raise:

Same as mode 1 but also raises z-order for floating windows.

7.2 Alt-Tab

Alt+Tab is a global hotkey intercepted BEFORE Ring 1 routing. It posts a focus-change command to Ring 0. The compositor handles it, not individual windows. Alt+Tab cycles through stack groups in most-recently-focused order.

8. Deskbar

8.1 Overview

The deskbar is a reserved overlay with three progressive disclosure states. It steals minimal screen real estate while always being accessible. The deskbar is ALWAYS the topmost element. Nothing draws above it. Nothing draws in its reserved band.

The deskbar is its own AILANG binary talking to the display server. It gets special treatment (always on top, reserved region, receives app-list events) but it's not baked into the compositor. Users can replace it.

8.2 Deskbar as Overlay

The deskbar's collapsed (thin) state defines a **hard boundary**. The BSP root region is permanently inset by the deskbar's thin size. This band is sacred — no window can ever occupy it.

Screen: 1920×1080, deskbar at top, thin=54px (5%)

Reserved deskbar band: y=0 to y=53 (HARD — nothing else ever here)

BSP root region: y=54 to y=1079 (all windows live here)

Deskbar hover overlay: y=0 to y=107 (reserved + 54px over BSP)

Deskbar expanded: y=0 to y=377 (reserved + deeper into BSP)

Hover and expanded states overlay into BSP territory. But since windows have hard boundaries and the BSP root starts at y=54, the underlying windows were always there — just temporarily occluded. When the deskbar collapses, a dirty region rebloit restores them. No relayout, no resize cascade.

8.3 Three States

State 0 — Thin (resting, 5% of screen dimension):

Visible 95% of the time. In terminal mode at 1080p: ~3 cell rows. Enough for:

 AILANG | TermA[3] TermB[1] Browser[2] Files | 14:32

App name, tab count in brackets, clock. Focused app highlighted. Click an app name to focus it without expanding.

State 1 — Hover (10%):

Mouse enters deskbar region, expands to double height. ~6 rows. Each app shows individual tab names:

 AILANG OS

▲ 14:32:07

▲ Mon Feb 9

▶ TermA [main.ailang] [build.log] [shell]

TermB [debug]

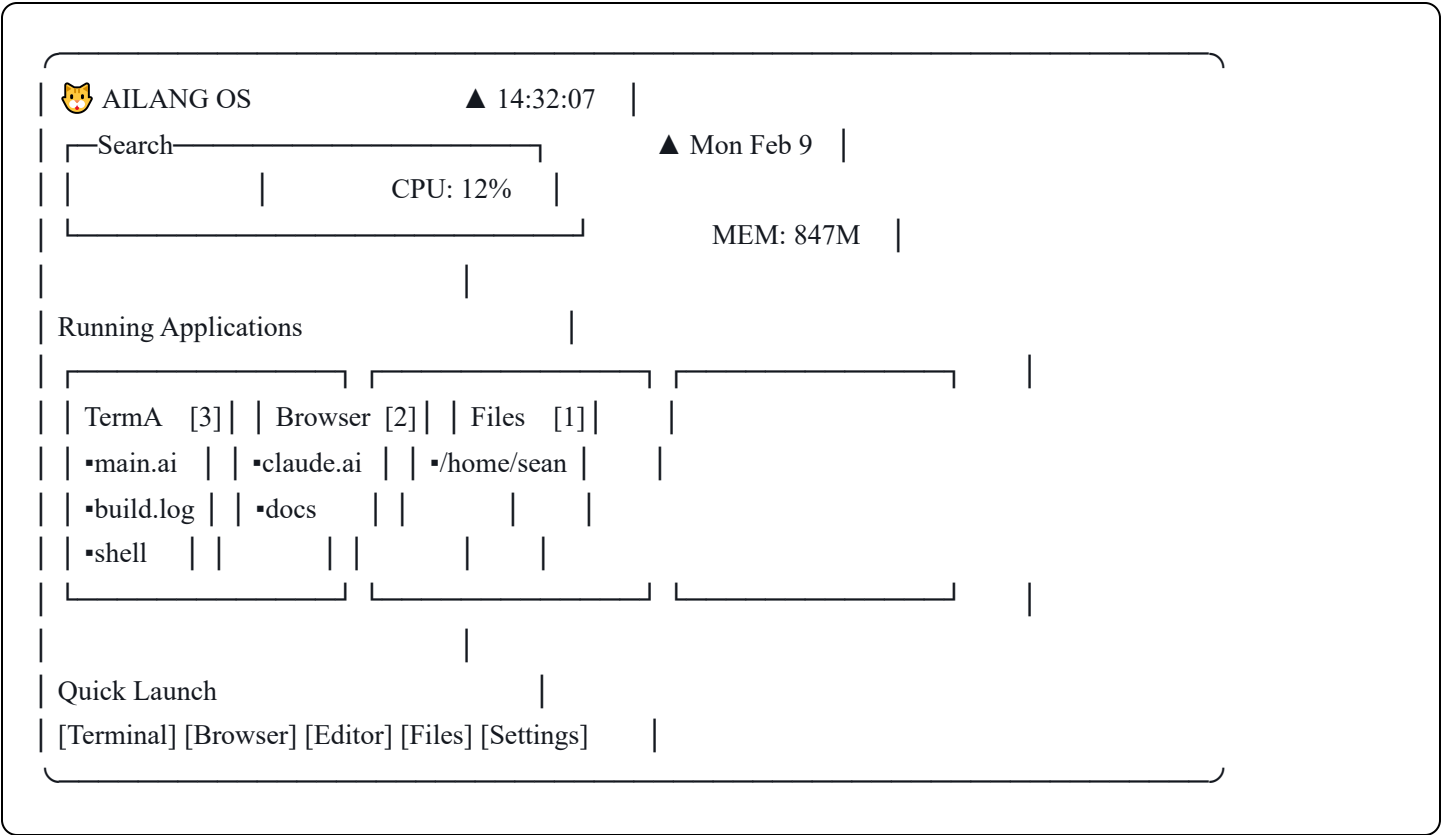
▶ Browser [claude.ai] [docs]

Files [/home/sean/src]

Focused app has  marker. Click specific tab to switch directly. Mouse leaves → collapse with 200ms delay.

State 2 — Expanded (30-40%):

Click deskbar or press hotkey. Full panel with search, system stats, app cards, quick launch:



Stays open until click outside, Escape, or app selection.

8.4 State Machine

```
FixedPool.DeskbarState {
  "THIN":    Initialize=0
  "HOVER":   Initialize=1
  "EXPANDED": Initialize=2
}
```

Transitions:

THIN + mouse_enter deskbar region → HOVER

HOVER + mouse_leave → delayed collapse (200ms timer)

HOVER + mouse_re_enter in delay → cancel collapse, stay HOVER

HOVER + click on deskbar → EXPANDED

EXPANDED + click outside → THIN (skip hover)

EXPANDED + Escape → THIN

EXPANDED + click app/tab → focus that app, then THIN

ANY + global hotkey (Alt+D) → toggle EXPANDED

8.5 Data Structures

```
FixedPool.DeskbarFields {
  "STATE":    Initialize=0
```

```

"POSITION":    Initialize=0  // TOP, BOTTOM, LEFT, RIGHT
"ZONE_ID":     Initialize=0  // which zone (-1 = span all)
"SURFACE":     Initialize=0  // deskbar surface handle
"THIN_SIZE":   Initialize=5  // percent of zone dimension
"HOVER_SIZE":  Initialize=10
"EXPANDED_SIZE": Initialize=35
"CURRENT_SIZE": Initialize=5  // current actual size
"TARGET_SIZE": Initialize=5  // animation target
"HOVER_TIMER": Initialize=0  // collapse delay timer
"APP_LIST":    Initialize=0  // pointer to running app table
"APP_COUNT":   Initialize=0
"FOCUSED_APP": Initialize=0  // index of focused app
"PINNED_APPS": Initialize=0  // quick launch list
"PINNED_COUNT": Initialize=0
"ANIM_MODE":   Initialize=0  // 0=snap, 1=slide
}

```

```

FixedPool.DeskbarAppEntry {
  "APP_ID":      Initialize=0
  "APP_NAME":    Initialize=0  // string pointer
  "STACK_GROUP": Initialize=0  // pointer to stack group
  "TAB_COUNT":   Initialize=0
  "TAB_NAMES":   Initialize=0  // array of tab name pointers
  "FOCUSED":     Initialize=0
  "PINNED":      Initialize=0
  "ICON":        Initialize=0  // surface handle (optional)
}

```

8.6 Animation

Terminal mode: snap (instant state change). Pixel mode: optional linear slide over 100-150ms.

During slide animation, `CURRENT_SIZE` interpolates toward `TARGET_SIZE` by a fixed step each frame. The expanding/collapsing strip is marked dirty each frame. When `CURRENT_SIZE == TARGET_SIZE`, animation completes.

9. Hard Boundaries — No Off-Screen Drawing

9.1 The Rule

Every surface has a hard rectangle. Nothing escapes it. No off-screen buffers. No windows hanging off monitor edges. No content bleeding past window borders. No drawing under the deskbar.

9.2 Enforcement

Every draw function gets a bounds check as the first operation:

```
// In EVERY draw operation:
IfCondition Or(LessThan(x, 0), Not(LessThan(x, surface_width))) ThenBlock: {
    ReturnValue(0) // out of bounds, no-op
}
IfCondition Or(LessThan(y, 0), Not(LessThan(y, surface_height))) ThenBlock: {
    ReturnValue(0) // out of bounds, no-op
}
```

No clipping stack. No clip regions. No scissor rectangles. The surface boundary IS the clip. Lines that would extend past the edge have endpoints clamped before the algorithm runs.

9.3 Scrollable Content — The App's Problem

The app wants scrolling? The app manages a viewport into its own data and draws only what's visible:

```
// Image editor: 5000×3000 image, 800×600 window
// App maintains viewport_x, viewport_y
Draw_BlitRect(
    window_surface,      // dest: the 800×600 surface
    0, 0,                // dest position: top-left
    image_data,          // src: app's own 5000×3000 data buffer
    viewport_x, viewport_y, // src position: viewport offset
    800, 600             // size: only what fits
)
```

The 5000×3000 data lives in app memory. It's not a surface. The compositor never knows about it. It sees an 800×600 surface, always completely filled.

9.4 Apps Don't Know Their Screen Position

The compositor creates the surface at BSP leaf dimensions and hands the handle to the app. The app draws at (0,0) through (width-1, height-1) in its own surface. The compositor knows where that surface lives on screen. The app is in a box and the box is its entire world.

App's perspective:

1. Request window from compositor
2. Receive surface handle + dimensions
3. Draw into surface at local coordinates
4. Never know screen position
5. Never draw outside surface bounds
6. Never think about other windows

9.5 Decorator Is Part of the Surface

The decorator (title bar, border, tabs) draws **INSIDE** the surface. The BSP leaf is the outer boundary. The decorator's `GetClientRect` returns the inner rectangle.

BSP leaf: 800×600

Catnip decorator: 1px border all sides, ~2 rows title/tab bar

Client rect: x=1, y=2, w=798, h=597

Decorator draws into surface[0..799, 0..599]

App draws into surface[1..798, 2..598]

Both within bounds. Neither exceeds 800×600.

10. Keyboard Shortcuts

10.1 Design Principles

- **Two-key maximum** (with one three-key exception for Shift+Tab)
- **Ctrl = app shortcuts** (copy, paste, save — forwarded to apps)
- **Alt = window management** (compositor intercepts ALL Alt+key combos)
- **Arrow keys for spatial operations**, letters for actions
- If you can't guess what it does in 5 seconds, it's wrong

10.2 Tier 1 — Universal App Shortcuts (Ctrl+key)

Forwarded to focused app. Compositor never intercepts these.

Ctrl+C	Copy
Ctrl+V	Paste
Ctrl+X	Cut
Ctrl+Z	Undo
Ctrl+Y	Redo
Ctrl+S	Save
Ctrl+A	Select all
Ctrl+F	Find
Ctrl+Q	Quit app
Ctrl+W	Close current tab in stack group
Ctrl+N	New window (app-specific)
Ctrl+Tab	Cycle tabs in stack group (forward)
Ctrl+Shift+Tab	Cycle tabs backward

10.3 Tier 2 — Window Management (Alt+key)

Intercepted by compositor. Apps never see Alt+key events.

Focus navigation (Alt+arrows):

Alt+Left	Focus window to the left
Alt+Right	Focus window to the right
Alt+Up	Focus window above
Alt+Down	Focus window below
Alt+Tab	Cycle stack groups (most-recently-focused order)

Tiling operations (Alt+letter):

Alt+H	Split current leaf horizontally (top/bottom)
Alt+V	Split current leaf vertically (left/right)
Alt+M	Merge — undo split, combine two leaves

Resize (Alt+Shift+arrows):

Alt+Shift+Left	Move split left (shrink from right)
Alt+Shift+Right	Move split right (expand right)
Alt+Shift+Up	Move split up (shrink from bottom)
Alt+Shift+Down	Move split down (expand down)

Stack operations (Alt+letter):

Alt+S + Arrow Stack with neighbor in arrow direction
 (two-step: Alt+S enters stack mode, arrow picks direction,
 Escape cancels)
Alt+P Pull — pop current tab out of stack to floating

General (Alt+letter):

Alt+D Toggle deskbar expanded
Alt+F Toggle floating (pull out of BSP / snap back in)
Alt+Enter Maximize (window takes entire zone, press again to restore)
Alt+Q Close window entirely
Alt+1..9 Focus Nth app in deskbar app list

10.4 Summary — 18 Total Combinations

Alt+Arrows (4) Focus navigation
Alt+Tab (1) Stack group cycling
Alt+H (1) Split horizontal
Alt+V (1) Split vertical
Alt+M (1) Merge split
Alt+Shift+Arrows (4) Resize splits
Alt+S+Arrow (1+dir) Stack with neighbor
Alt+P (1) Pull from stack
Alt+D (1) Deskbar toggle
Alt+1..9 (9) Quick app focus
Alt+Q (1) Close window
Alt+F (1) Float toggle
Alt+Enter (1) Maximize toggle

10.5 Learning Curve

Day 1 (basics):

Alt+Arrows "move focus to another window"
Alt+Tab "flip through my stacks"
Alt+D "open the menu"
Ctrl+C/V/X "copy paste like always"

Day 2 (tiling):

Alt+H / Alt+V "split my screen"
Alt+M "unsplit"
Alt+Shift+Arrows "make a window bigger/smaller"

Day 3 (power user):

Alt+S + direction	"group these windows"
Alt+P	"ungroup this tab"
Alt+1..9	"jump to app by number"
Alt+Enter	"fullscreen this"

10.6 Alt+S Stack Mode State Machine

Alt+S enters a transient "stack mode" in the compositor. The next arrow key determines direction. Escape cancels.

Ring 0 receives STACK_BEGIN:

- set compositor.stack_pending = 1
- next Ring 1 arrow key:
 - resolve direction, find neighbor BSP leaf
 - execute stack operation
 - clear stack_pending
- next Ring 1 Escape:
 - clear stack_pending, cancel

10.7 Hotkey Table

Hotkeys are configurable. The table is a flat array of {keycode, modifier_mask, action} tuples. At 18 entries, linear scan is faster than any hash.


```
FixedPool.HotkeyEntry {  
    "KEYCODE":    Initialize=0  
    "MODIFIERS":  Initialize=0 // ALT, ALT_SHIFT, etc.  
    "ACTION":     Initialize=0 // HotkeyAction enum value  
}
```

```
FixedPool.HotkeyTableConfig {  
    "MAX_ENTRIES": Initialize=32 // room to grow  
    "ENTRY_SIZE":  Initialize=24 // 3 fields × 8 bytes  
}
```

```
FixedPool.HotkeyAction {  
    "FOCUS_LEFT":    Initialize=1  
    "FOCUS_RIGHT":   Initialize=2  
    "FOCUS_UP":      Initialize=3  
    "FOCUS_DOWN":    Initialize=4  
    "TAB_NEXT":      Initialize=5  
    "TAB_PREV":      Initialize=6  
    "SPLIT_H":       Initialize=10  
    "SPLIT_V":       Initialize=11  
    "MERGE":         Initialize=12  
    "RESIZE_LEFT":   Initialize=20  
    "RESIZE_RIGHT":  Initialize=21  
    "RESIZE_UP":     Initialize=22  
    "RESIZE_DOWN":   Initialize=23  
    "STACK_BEGIN":   Initialize=30  
    "PULL_TAB":      Initialize=31  
    "FLOAT_TOGGLE":  Initialize=32  
    "MAXIMIZE":      Initialize=33  
    "CLOSE_WINDOW":  Initialize=40  
    "DESKBAR_TOGGLE": Initialize=50  
    "APP_FOCUS_1":   Initialize=51  
    "APP_FOCUS_2":   Initialize=52  
    "APP_FOCUS_3":   Initialize=53  
    "APP_FOCUS_4":   Initialize=54  
    "APP_FOCUS_5":   Initialize=55  
    "APP_FOCUS_6":   Initialize=56  
    "APP_FOCUS_7":   Initialize=57  
    "APP_FOCUS_8":   Initialize=58  
    "APP_FOCUS_9":   Initialize=59  
}
```

10.8 Input Interception

```
// In Ring 1 input dispatch:
IfCondition And(event.type == KEY_DOWN, modifier_alt_held) ThenBlock: {
    action = HotkeyTable_Lookup(event.keycode)
    IfCondition NotEqual(action, 0) ThenBlock: {
        Ring0_Post(action)
        ReturnValue(1) // consumed, do NOT forward to app
    }
}
// Alt not held or no match: forward to focused app
```

11. File Hierarchy

```
Libraries/
├── Display/
│   │
│   └── README.md
│   │
│   └── Core/                                # AXIS 0: Surface
│       ├── Library.DSurfaceTypes.ailang
│       ├── Library.DSurface.ailang
│       └── Library.DSurfaceTable.ailang
│       │
│       └── Draw/                            # AXIS 1: Drawing
│           ├── Library.DDrawTypes.ailang
│           ├── Library.DDrawDispatch.ailang
│           ├── Library.DDrawCell.ailang
│           ├── Library.DDrawPixel.ailang
│           └── Library.DDrawText.ailang
│           │
│           └── Compose/                     # AXIS 2: Compositor
│               ├── Library.DComposeTypes.ailang
│               ├── Library.DCompose.ailang    # BSP region map
│               ├── Library.DComposeBSP.ailang # BSP tree operations
│               ├── Library.DComposeStack.ailang # Stack group management
│               ├── Library.DComposeFloat.ailang # Floating window list
│               ├── Library.DComposeDirty.ailang # Dirty region tracking
│               └── Library.DComposeDiff.ailang # Frame diff (terminal)
```

		└─ Ring/	# EVENT ARCHITECTURE
		└─ Library.DRingTypes.ailang	# Ring buffer struct, entry types
		└─ Library.DRing.ailang	# Ring_Post / Ring_Drain
		└─ Library.DRing0.ailang	# Focus/Control ring
		└─ Library.DRing1.ailang	# Input Events ring
		└─ Library.DRing2.ailang	# Draw Commands ring
		└─ Library.DRing3.ailang	# Output/Dirty ring
		└─ Input/	# AXIS 3: Input
		└─ Library.DInputTypes.ailang	
		└─ Library.DInput.ailang	
		└─ Library.DInputTerm.ailang	
		└─ Library.DInputEvdev.ailang	
		└─ Library.DHotkey.ailang	# Hotkey table + interception
		└─ Layout/	# AXIS 4: Layout
		└─ Library.DLayoutTypes.ailang	
		└─ Library.DLayout.ailang	
		└─ Library.DLayoutVBox.ailang	
		└─ Library.DLayoutHBox.ailang	
		└─ Library.DLayoutGrid.ailang	
		└─ Library.DLayoutDock.ailang	
		└─ Library.DLayoutTile.ailang	# Shared-edge tiling constraints
		└─ Widget/	# AXIS 5: Widgets (OPTIONAL)
		└─ Library.DWidgetTypes.ailang	
		└─ Library.DWidget.ailang	
		└─ Library.DWidgetLabel.ailang	
		└─ Library.DWidgetButton.ailang	
		└─ Library.DWidgetTextInput.ailang	
		└─ Library.DWidgetList.ailang	
		└─ Library.DWidgetPanel.ailang	
		└─ Library.DWidgetCanvas.ailang	
		└─ Decorator/	# Window Chrome
		└─ Library.DDecoratorTypes.ailang	
		└─ Library.DDecorator.ailang	
		└─ Library.DDecoratorNone.ailang	
		└─ Library.DDecoratorMinimal.ailang	
		└─ Library.DDecoratorClassic.ailang	
		└─ Library.DDecoratorCatnip.ailang	
		└─ Window/	# Window Management
		└─ Library.DWindowTypes.ailang	

```

| | └─ Library.DWindow.ailang
| |
| | └─ Zone/                                # Multi-Monitor
| |   └─ Library.DZoneTypes.ailang          # Zone struct, surface modes
| |   └─ Library.DZone.ailang              # Zone_Add/Remove/Configure
| |   └─ Library.DZoneScale.ailang         # DPI scaling per zone
| |
| | └─ Deskbar/                             # Deskbar
| |   └─ Library.DDeskbarTypes.ailang
| |   └─ Library.DDeskbar.ailang           # State machine, transitions
| |   └─ Library.DDeskbarRender.ailang     # Three-state rendering
| |   └─ Library.DDeskbarAppList.ailang    # Running app tracking
| |
| | └─ Render/                             # Output Backends
| |   └─ Library.DRenderTerm.ailang
| |   └─ Library.DRenderFB.ailang
| |   └─ Library.DRenderDRM.ailang
| |
| | └─ Font/
| |   └─ Library.DFont.ailang
| |
| | └─ Server/                             # Display Server
| |   └─ Library.DServerTypes.ailang
| |   └─ Library.DServer.ailang
| |   └─ Library.DServerProtocol.ailang
| |   └─ Library.DClient.ailang
| |
| └─ Display.ailang                        # Top-level entry point
|
└─ Library.TUI.ailang                     # EXISTING
└─ Library.Framebuffer.ailang             # EXISTING
└─ Library.Font.ailang                    # EXISTING
└─ Library.FontPSF.ailang                 # EXISTING
└─ Library.FontBitmap.ailang              # EXISTING
└─ Library.XArrays.ailang                 # EXISTING

```

11.1 File Count

```

Core:    3 files
Draw:    5 files
Compose: 7 files (+3 from v1: BSP, Stack, Float)
Ring:    6 files (NEW)
Input:   5 files (+1: Hotkey)

```

Layout: 7 files (+1: Tile constraints)

Widget: 8 files

Decorator: 6 files

Window: 2 files

Zone: 3 files (NEW)

Deskbar: 4 files (NEW)

Render: 3 files

Font: 1 file

Server: 4 files

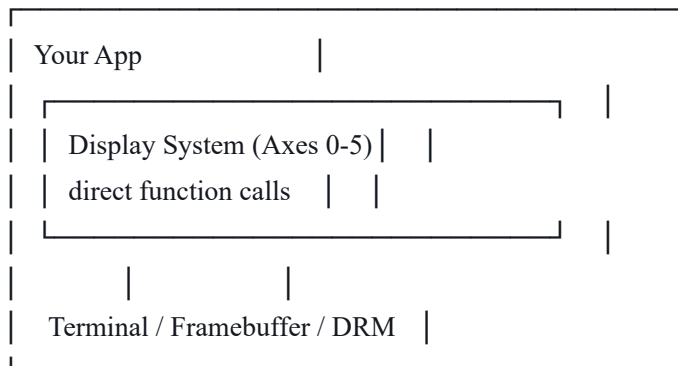
Top-level: 1 file

Total: 65 files

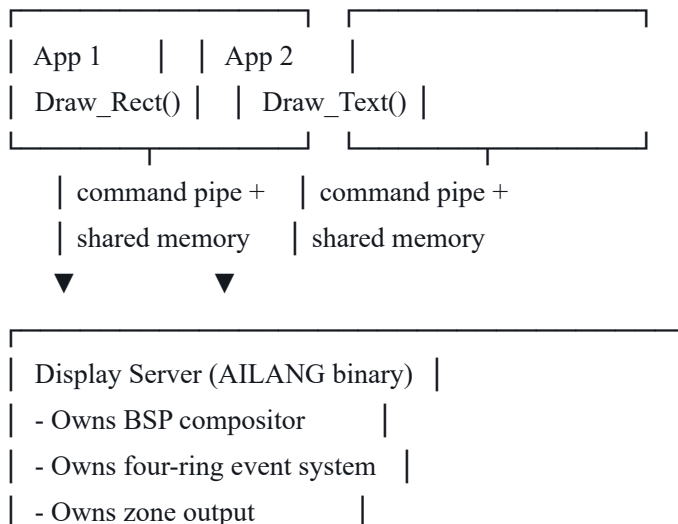
12. Display Server Architecture

12.1 Two Modes

MODE 1: Standalone (single binary, no IPC)



MODE 2: Display Server (multi-process)



- | | |
|------------------------------|--|
| - Manages stack groups | |
| - Runs deskbar (or separate) | |
| | |
| Terminal / Framebuffer / DRM | |

12.2 IPC Mechanism

- **Surface buffers:** shared memory (mmap + fd passing) → zero-copy
- **Command stream:** Unix domain socket → client sends command structs (Ring 2)
- **Events:** reverse channel → server sends input events to clients (Ring 1)
- **Sync:** dirty flags in shared memory header → no round-trip for most frames

12.3 Server Main Loop

While running:

1. Accept new client connections
2. Drain Ring 0 (focus/control/BSP changes)
3. Drain Ring 1 (input from devices, route to clients)
4. Process Ring 2 per client (draw commands, round-robin)
5. Process Ring 3 (dirty regions → composite → zone flush)
6. Update deskbar if app list changed
7. Yield / sleep until next event or frame deadline

13. Implementation Phases

Phase 1: Surface + Draw Terminal (~800 LOC)

Files: DSurfaceTypes, DSurface, DSurfaceTable, DDrawTypes, DDrawDispatch, DDrawCell

Test: Draw boxes, text, lines on terminal with hard boundary enforcement

Phase 2: Surface + Draw Pixel (~400 LOC)

Files: DDrawPixel (wrapping existing FB_* functions)

Test: Same draw calls rendering to framebuffer

Phase 3: Ring Buffers (~400 LOC)

Files: DRingTypes, DRing, DRing0, DRing1, DRing2, DRing3

Test: Post/drain on all four rings, overflow policies

Phase 4: BSP Compositor (~800 LOC)

Files: DComposeTypes, DCompose, DComposeBSP, DComposeStack, DComposeFloat,
DComposeDirty, DComposeDiff

Test: BSP split/merge, stack groups, region ownership, hit testing

Phase 5: Zones + Render (~500 LOC)

Files: DZoneTypes, DZone, DZoneScale, DRenderTerm, DRenderFB

Test: Multi-zone output, DPI scaling, combined vs zoned modes

Phase 6: Input + Hotkeys (~600 LOC)

Files: DInputTypes, DInput, DInputTerm, DInputEvdev, DHotkey

Test: Event routing through rings, hotkey interception, focus modes

Phase 7: Window + Decorator + Layout (~800 LOC)

Files: DWindowTypes, DWindow, DDecoratorTypes, DDecorator,
DDecoratorNone/Minimal/Classic/Catnip,
DLayoutTypes, DLayout, DLayoutVBox/HBox/Grid/Dock/Tile

Test: Window_Create with descriptor → decorated tiled window, tab switching

Phase 8: Deskbar (~500 LOC)

Files: DDeskbarTypes, DDeskbar, DDeskbarRender, DDeskbarAppList

Test: Three-state transitions, app list display, hotkey toggle

Phase 9: Widgets (~500 LOC)

Files: DWidgetTypes, DWidget, DWidgetLabel/Button/TextInput/List/Panel/Canvas

Test: Interactive widgets in tiled windows

Phase 10: Display Server (~800 LOC)

Files: DServerTypes, DServer, DServerProtocol, DClient, DFont, Display.ailang

Test: Two separate AILANG binaries sharing a display via server

Total Estimated Size

Phase 1-2: ~1,200 LOC (draw anything)

Phase 3-4: ~1,200 LOC (ring events + BSP compositor)

Phase 5-6: ~1,100 LOC (zones + input + hotkeys)

Phase 7-8: ~1,300 LOC (windows + decorators + deskbar)

Phase 9: ~ 500 LOC (widgets)

Phase 10: ~ 800 LOC (multi-process server)

Total: ~6,100 LOC

For comparison:

AILANG compiler (self-hosting): ~15,000+ LOC

Qt5 core + widgets: ~2,000,000 LOC

Weston (Wayland compositor): ~60,000 LOC

This project: ~6,100 LOC, 65 files

14. Usage Examples

14.1 Just Draw (Axes 0+1 only)

```
ailang

SubRoutine.Main {
    Body: {
        TUI_Init()
        surf = Surface_Create(80, 24, SurfaceFormat.CELL_32)
        Draw_FillRect(surf, 0, 0, 80, 24, 0x000040)
        Draw_Rect(surf, 5, 2, 30, 10, 0xFFFFFFFF)
        Draw_Text(surf, 7, 4, "Hello!", 0, 0x00FF00, -1)
        RenderTerm_Flush(surf)
        TUI_GetKey()
        TUI_Shutdown()
    }
}
```


14.2 Windowed App (Stack+Tile)

```
ailang

SubRoutine.Main {
  Body: {
    Display_Init(0)  // 0=terminal

    // Create window — auto-stacks with focused group
    desc = Allocate(WinDesc.SIZE)
    StoreValue(Add(desc, WinDesc.DECORATOR_ID), DecoratorID.CATNIP)
    StoreValue(Add(desc, WinDesc.TITLE_PTR), "My App")
    win = Window_Create(desc)

    lbl = Widget_Label("Welcome to AILANG Display!")
    btn = Widget_Button("[ OK ]", AddressOf(OnClick))
    Window_AddWidget(win, lbl)
    Window_AddWidget(win, btn)
    Window_Show(win)

    Display_Run()  // blocks: Ring1→Ring0→Ring2→Ring3 tick loop
  }
}
```

14.3 Custom Canvas in a Tiled Window

```
ailang

Function.MyDraw {
  Input: widget: Integer
  Input: surface: Integer
  Body: {
    // Draw within surface bounds — hard boundary enforced
    w = Surface_GetWidth(surface)
    h = Surface_GetHeight(surface)
    Draw_FillRect(surface, 0, 0, w, h, 0x001020)
    Draw_Circle(surface, Divide(w, 2), Divide(h, 2), 30, 0xFF4400)
    Draw_Text(surface, 10, Subtract(h, 20), "Custom!", 0, 0xFFFFFFFF, -1)
  }
}
```

15. The Full Vision

Today: App → links Qt (500MB) → links X11 → links GL → kernel
Build: cmake + moc + pkg-config + 47 headers + pray
Desktop: 4 virtual desktops, 47 hidden windows, alt-tab roulette
Compositing: alpha blending everything for drop shadows nobody needs

AILANG: App → Display_*() calls → Display Server → syscalls → kernel
Build: ailang compile myapp.ailang → done
Desktop: ONE desktop, stack+tile, deskbar shows everything
Compositing: memcpy per dirty region, zero blending

No headers. No linker flags. No build system. No ABI compatibility issues.
No transparency. No virtual desktops. No off-screen windows.

Hard boundaries. Opaque surfaces. BSP tiling. Stack groups.
One desktop to rule them all.

Appendix A: Comparison With Existing Systems

Feature	AILANG Display	X11/Wayland	Qt/GTK	Haiku
Compositing	opaque memcpy	alpha blend	alpha blend	opaque
Transparency	none (by design)	full	full	none
Window model	BSP stack+tile	free float	free float	stack+tile
Virtual desktops	NO (one desktop)	yes	N/A	optional
Off-screen draw	NO (hard bounds)	yes	yes	yes
Event model	4-ring buffers	X event queue	signal/slot	BMessage
Focus policy	pluggable 3-mode	configurable	toolkit	click-to-focus
Multi-monitor	unified+zones	Xrandr/Wayland	toolkit	per-screen
DPI scaling	per-zone integer	per-output	per-widget	global
IPC	shm+unix socket	X protocol	N/A	BMessage+port
Code size	~6,100 LOC	millions	millions	~100K
Build dependency	none (self-host)	massive	massive	moderate

Document version: 2.0 Incorporates: synchronization architecture, four-ring events, BSP tiling, stack groups, deskbar design, hard boundaries, keyboard shortcuts, multi-monitor zones, DPI scaling, no-transparency

